ABSTRACT

This research presents a novel Facial Detection and Attendance System implemented in the Python programming language. The proposed system aims to automate the process of capturing and analyzing facial features for attendance tracking purposes. By leveraging state-of-the-art computer vision algorithms and machine learning techniques, this system offers a reliable, efficient, and user-friendly solution for various educational, corporate, and institutional settings.

The system's core functionality involves detecting and recognizing individual faces in real-time using advanced facial recognition algorithms. It integrates cutting-edge technologies such as Haar Cascades to accurately identify and authenticate users. Additionally, it incorporates a user-friendly graphical user interface (GUI) for seamless interaction and ease of use.

The Facial Detection and Attendance System's key benefits include minimizing human error, enhancing security, and streamlining the attendance management process. Furthermore, the system's open-source nature allows for easy customization and integration with existing infrastructure.

In conclusion, this Python-based Facial Detection and Attendance System offers a robust and efficient solution for automated attendance tracking, paving the way for a more advanced and technologically driven future in various sectors.

Table of Contents

List of Figures

CHAPTER 1
INTRODUCTON

1.2 OBJECTIVE AND PROBLEM STATEMENT

Objective:

The primary objective of the FACE DETECTION ATTENDANCE SYSTEM project in Python language is to develop an automated and efficient solution for tracking and recording the attendance of individuals in an organization or educational institution. This system aims to simplify the attendance-taking process and enhance accuracy by utilizing face detection technology.

Problem Statement:

The primary objective of this project is to design and develop an automated Face Detection Attendance System that utilizes advanced facial recognition technology to track and manage students' attendance in an organization. The system should be capable of accurately detecting and identifying individuals based on their unique facial features, ensuring secure and reliable attendance monitoring.

Key Features and Functionalities:

1. Facial Recognition: The system should be equipped with a robust facial recognition algorithm that can accurately identify students by analyzing their facial features.
2. User Registration: A user-friendly interface should be developed to allow students to register their profiles by providing essential details and capturing their facial images.
3. Real-time Notifications: The system should send real-time notifications to students and concerned authorities regarding their attendance status.

4. Scalability and Integration: The Face Detection Attendance System should be designed to accommodate future expansion, including the addition of new students.
5. Text to speech: Project gives voice message to guide user.

By addressing these features and functionalities, the Face Detection Attendance System in Python language aims to provide an innovative, efficient, and secure solution for managing students' attendance in organizations.

CHAPTER 2
METHODOLOGY

2.1 ALGORITHMIC DETAILS

1. Choose a suitable programming language and libraries: Select Python as the programming language and consider using OpenCV for its computer vision capabilities, including face detection.

2. Install the required libraries and dependencies: Download and install OpenCV for Python, ensuring it is compatible with your Python version.

3. Prepare the development environment: Set up your Integrated Development Environment (IDE) like VSCODE for writing and compiling the Python code.

4. Create a new Python project: In your IDE, create a new Python project for the Face Detection Attendance System.

5. Design the user interface: Decide on the user interface components required for the system, such as buttons, text fields, and labels.

6. Import necessary classes and libraries: Include the required Python classes and OpenCV libraries in your project.

7. Load the pre-trained Haar Cascade Classifier: Obtain the Haar Cascade XML file for face detection and load it into your program.

8. Capture video input: Use the VideoCapture class from OpenCV to capture video input from a webcam or any other video source.

9. Process frames and detect faces: Iterate through each frame of the captured video, resize the frame, and apply the face detection algorithm.

10. Implement attendance tracking: Store student information in a TrainingImage folder and train the model with each student's face with Trainner.yml model.

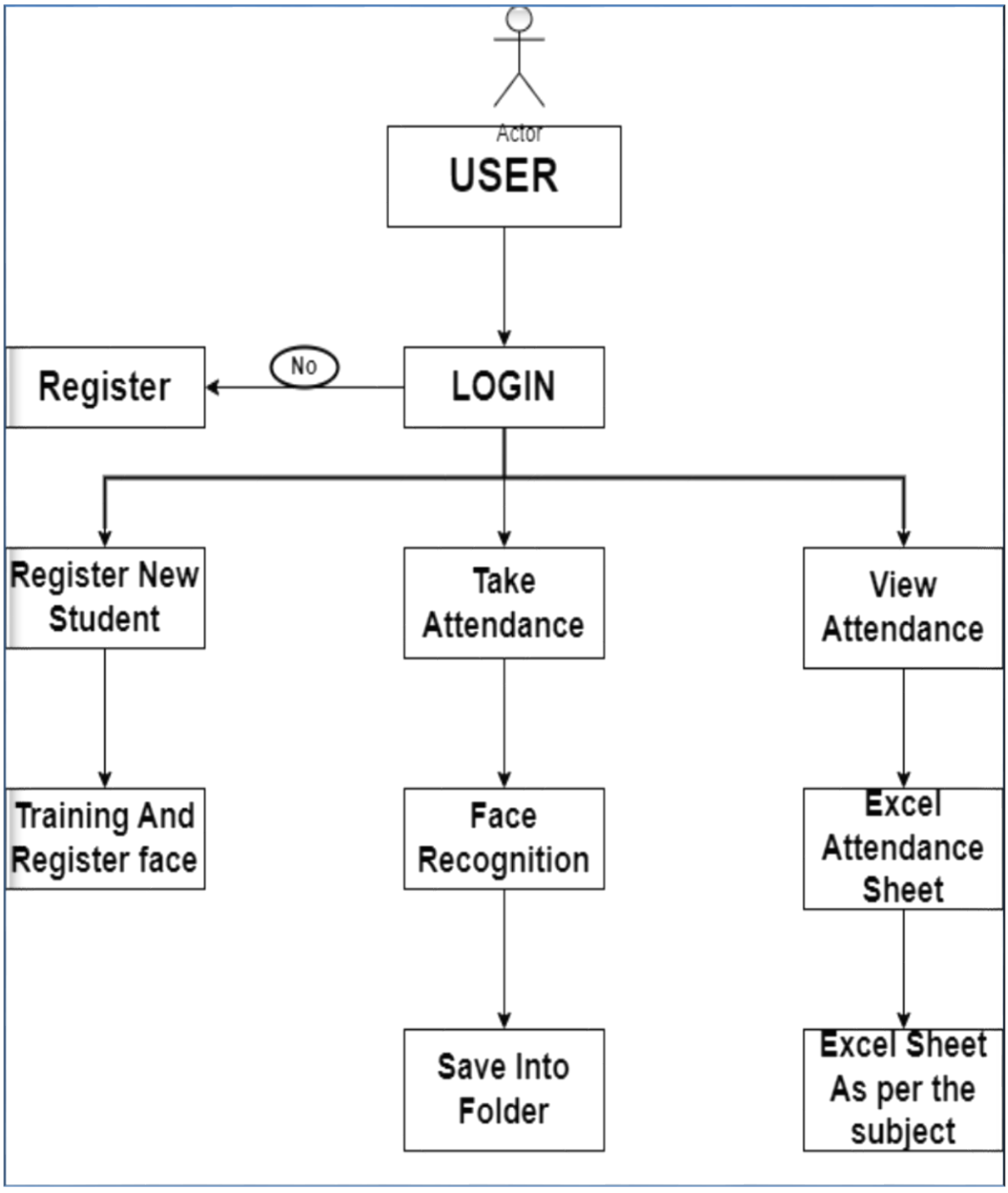
11. Compare detected faces with stored student faces: For each detected face, check if it matches any of the stored student faces to identify the attending student.

12. Mark attendance: Once a student is identified, update the attendance record accordingly.

13. Display relevant information: Show the detected face and the corresponding student's name on the user interface.

14. Continuously update the system: Keep the program running to continuously detect faces and update attendance in real-time in subject excel sheet.

Flowchart: -


Actor
USER
LOGIN
No
Register
Register New Student
Take Attendance
View Attendance
Training And Register face
Face Recognition
Excel Attendance Sheet
Save Into Folder
Excel Sheet As per the subject

2.2 HARDWARE AND SOFTWARE REQUIREMENTS

2.2.1 HARDWARE REQUIREMENTS

1. RAM: 512 MB RAM
2. Hard Drive: 40 GB Hard Drive
3. Processor: Intel Core 2 Processor
4. Camera module (Webcam)

2.2.2 SOFTWARE REQUIREMENTS

1.IDE: VSCODE IDE for Python Developers

2.Python version (Recommended): Python 3.1+

3.Database: MySQL

4.Type: Desktop Application

2.3 DESIGN DETAILS:

A Face Detection Attendance System is an automated solution that uses facial recognition technology to track and manage the attendance of individuals in an organization or educational institution. In this case, you are looking for a design detail for implementing this system in Python language. Here's a general outline of the system's design:

- **Algorithms Used in Project: -**

1. LBPHFaceRecognizer

The LBPH algorithm works as follows:

Local Binary Patterns (LBP): The algorithm first extracts local features from the face images by applying the LBP operator. The LBP operator converts each pixel in the image into a binary number based on the comparison of the pixel's value with its neighbouring pixels.

Histogram: The algorithm then creates a histogram of the LBP values for the entire face image. This histogram represents the distribution of the local features in the face.

Training: During the training phase, the algorithm learns the LBPH histograms for each known face in the training dataset.

Recognition: During the recognition phase, the algorithm extracts the LBPH histogram of the input face image and compares it with the stored histograms of the known faces. The face with the closest matching histogram is identified as the recognized face.

The LBPH algorithm is known for its robustness to variations in lighting, pose, and expression, making it a popular choice for real-time face recognition applications. It is also relatively computationally efficient compared to other face recognition algorithms, making it suitable for embedded systems and mobile devices.

2. Haarcascade Algorithm

The haarcascade_frontalface_default.xml file is a pre-trained Haar Cascade Classifier used in this script for face detection. Here's a breakdown of the algorithm and its usage in the code:

Haarcascade Algorithm Features:

Haar-like Features: The Haar Cascade Classifier uses Haar-like features, which are simple rectangular features that can be used to detect objects in an image. These features are based on the differences in the sum of pixel intensities between adjacent rectangular regions.

AdaBoost: The Haar Cascade Classifier uses the AdaBoost machine learning algorithm to train a strong classifier from a set of weak classifiers (Haar-like features). AdaBoost iteratively selects the best weak classifiers and combines them to create a strong classifier.

Cascade of Classifiers: The Haar Cascade Classifier is a cascade of multiple stages of classifiers. Each stage is trained to detect the target object (in this case, a face) using a set of Haar-like features. The cascade structure allows for efficient object detection, as the image is only processed by the subsequent stages if it passes the previous stages.

CHAPTER 3

IMPLEMENTATION AND RESULTS

3.1 IMPLEMENATAION:

trainImage.py - E:\Softwares\VScode\GitHub\3 option\Attendance-Management-system-using-face-recognition-master\trainImage.py (3.12.1)

File Edit Format Run Options Window Help

```
import csv
import os, cv2
import numpy as np
import pandas as pd
import datetime
import time
from PIL import ImageTk, Image


# Train Image
def TrainImage(haarcasecade_path, trainimage_path, trainimagelabel_path, message,text_to_speech):
    recognizer = cv2.face.LBPHFaceRecognizer_create()
    detector = cv2.CascadeClassifier(haarcasecade_path)
    faces, Id = getImagesAndLables(trainimage_path)
    recognizer.train(faces, np.array(Id))
    recognizer.save(trainimagelabel_path)
    res = "Image Trained successfully"  # +",".join(str(f) for f in Id)
    message.configure(text=res)
    text_to_speech(res)


def getImagesAndLables(path):
    # imagePath = [os.path.join(path, f) for d in os.listdir(path) for f in d]
    newdir = [os.path.join(path, d) for d in os.listdir(path)]
    imagePath = [
        os.path.join(newdir[i], f)
        for i in range(len(newdir))
        for f in os.listdir(newdir[i])
    ]
    faces = []
    Ids = []
    for imagePath in imagePath:
        pilImage = Image.open(imagePath).convert("L")
        imageNp = np.array(pilImage, "uint8")
        Id = int(os.path.split(imagePath)[-1].split("_")[1])
        faces.append(imageNp)
        Ids.append(Id)
    return faces, Ids
```

Fig 1: - Training Images for taking all the images from the folder given as path in method

automaticAttedance.py - E:\Softwares\VScode\GitHub\3 option\Attendance-Management-system-using-face-recognition-master\automaticAttedance.py (3.12.1)

File Edit Format Run Options Window Help

```
import tkinter as tk
from tkinter import *
import os, cv2
import shutil
import csv
import numpy as np
from PIL import ImageTk, Image
import pandas as pd
import datetime
import time
import tkinter.ttk as tkk
import tkinter.font as font

haarcasecade_path = "E:\\Softwares\\VScode\\GitHub\\3 option\\Attendance-Management-system-using-face-recognition-master\\haarcascade_frontalface_default.xml"
trainimagelabel_path = (
    "E:\\Softwares\\VScode\\GitHub\\3 option\\Attendance-Management-system-using-face-recognition-master\\TrainingImageLabel\\Trainner.yml"
)
trainimage_path = "E:\\Softwares\\VScode\\GitHub\\3 option\\Attendance-Management-system-using-face-recognition-master\\TrainingImage"
studentdetail_path = (
    "E:\\Softwares\\VScode\\GitHub\\3 option\\Attendance-Management-system-using-face-recognition-master\\StudentDetails\\studentdetails.csv"
)
attendance_path = "E:\\Softwares\\VScode\\GitHub\\3 option\\Attendance-Management-system-using-face-recognition-master\\Attendance"
# for choose subject and fill attendance
def subjectChoose(text_to_speech):
    def FillAttendance():
        sub = tx.get()
        now = time.time()
        future = now + 20
        print(now)
        print(future)
        if sub == "":
            t = "Please enter the subject name!!!"
            text_to_speech(t)
        else:
            try:
                recognizer = cv2.face.LBPHFaceRecognizer_create()
                try:
                    recognizer.read(trainimagelabel_path)
                except:
                    e = "Model not found,please train model"
                    Notifica.configure(
                        text=e,
                        bg="black",
                        fg="yellow",
                        width=33,
                        font=("times", 15, "bold"),
```

```
        )
        Notifica.place(x=20, y=250)
        text_to_speech(e)
    facecasCade = cv2.CascadeClassifier(haarcasecade_path)
    df = pd.read_csv(studentdetail_path)
    cam = cv2.VideoCapture(0)
    font = cv2.FONT_HERSHEY_SIMPLEX
    col_names = ["Enrollment", "Name"]
    attendance = pd.DataFrame(columns=col_names)
    while True:
        ___, im = cam.read()
        gray = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)
        faces = facecasCade.detectMultiScale(gray, 1.2, 5)
        for (x, y, w, h) in faces:
            global Id

            Id, conf = recognizer.predict(gray[y : y + h, x : x + w])
            if conf < 70:
                print(conf)
                global Subject
                global aa
                global date
                global timeStamp
                Subject = tx.get()
                ts = time.time()
                date = datetime.datetime.fromtimestamp(ts).strftime(
                    "%Y-%m-%d"
                )
                timeStamp = datetime.datetime.fromtimestamp(ts).strftime(
                    "%H:%M:%S"
                )
                aa = df.loc[df["Enrollment"] == Id]["Name"].values
                global tt
                tt = str(Id) + "-" + aa
                # En='1604501160'+str(Id)
                attendance.loc[len(attendance)] = [
                    Id,
                    aa,
                ]
                cv2.rectangle(im, (x, y), (x + w, y + h), (0, 260, 0), 4)
                cv2.putText(
                    im, str(tt), (x + h, y), font, 1, (255, 255, 0,), 4
                )
            else:
                Id = "Unknown"
                tt = str(Id)
```

```
                # En='1604501160'+str(Id)
                attendance.loc[len(attendance)] = [
                    Id,
                    aa,
                ]
                cv2.rectangle(im, (x, y), (x + w, y + h), (0, 260, 0), 4)
                cv2.putText(
                    im, str(tt), (x + h, y), font, 1, (255, 255, 0,), 4
                )
            else:
                Id = "Unknown"
                tt = str(Id)
                cv2.rectangle(im, (x, y), (x + w, y + h), (0, 25, 255), 7)
                cv2.putText(
                    im, str(tt), (x + h, y), font, 1, (0, 25, 255), 4
                )
        if time.time() > future:
            break

        attendance = attendance.drop_duplicates(
            ["Enrollment"], keep="first"
        )
        cv2.imshow("Filling Attendance...", im)
        key = cv2.waitKey(30) & 0xFF
        if key == 27:
            break

    ts = time.time()
    print(aa)
    # attendance["date"] = date
    # attendance["Attendance"] = "P"
    attendance[date] = 1
    date = datetime.datetime.fromtimestamp(ts).strftime("%Y-%m-%d")
    timeStamp = datetime.datetime.fromtimestamp(ts).strftime("%H:%M:%S")
    Hour, Minute, Second = timeStamp.split(":")
    # fileName = "Attendance/" + Subject + ".csv"
    path = os.path.join(attendance_path, Subject)
    fileName = (
        f"{path}/"
        + Subject
        + "_"
        + date
        + "_"
        + Hour
        + "-"
        + Minute
        + "-"
```

```
        + Second
        + ".csv"
    )
    attendance = attendance.drop_duplicates(["Enrollment"], keep="first")
    print(attendance)
    attendance.to_csv(fileName, index=False)

    m = "Attendance Filled Successfully of " + Subject
    Notifica.configure(
        text=m,
        bg="black",
        fg="yellow",
        width=33,
        relief=RIDGE,
        bd=5,
        font=("times", 15, "bold"),
    )
    text_to_speech(m)

    Notifica.place(x=20, y=250)

    cam.release()
    cv2.destroyAllWindows()
```

Fig 2: - Code to Recognize Face from Trained model of .yml file and make entry in the excel sheet.

3.2 RESULTS:

Fig 3: Registration Page

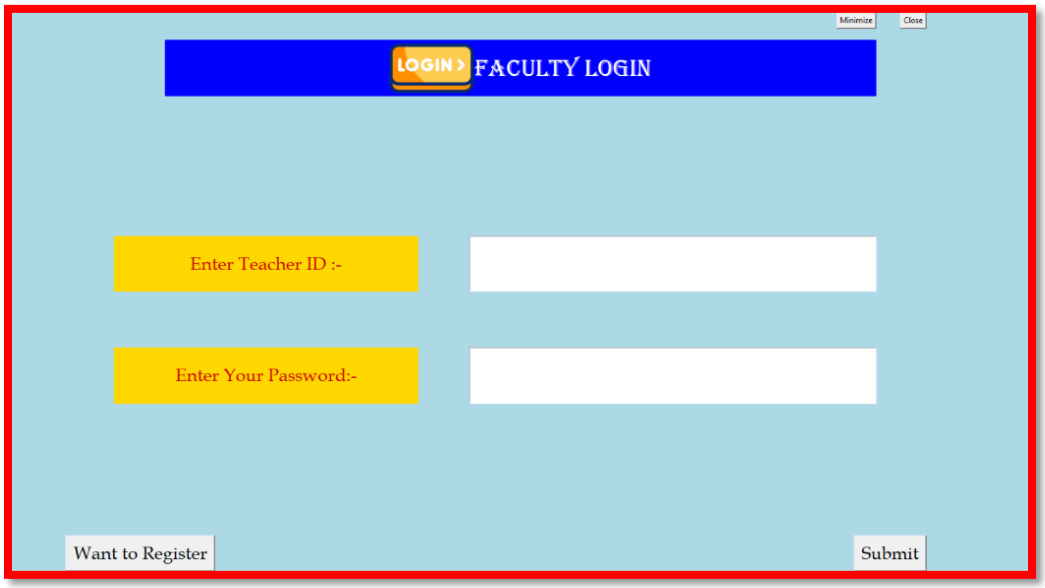
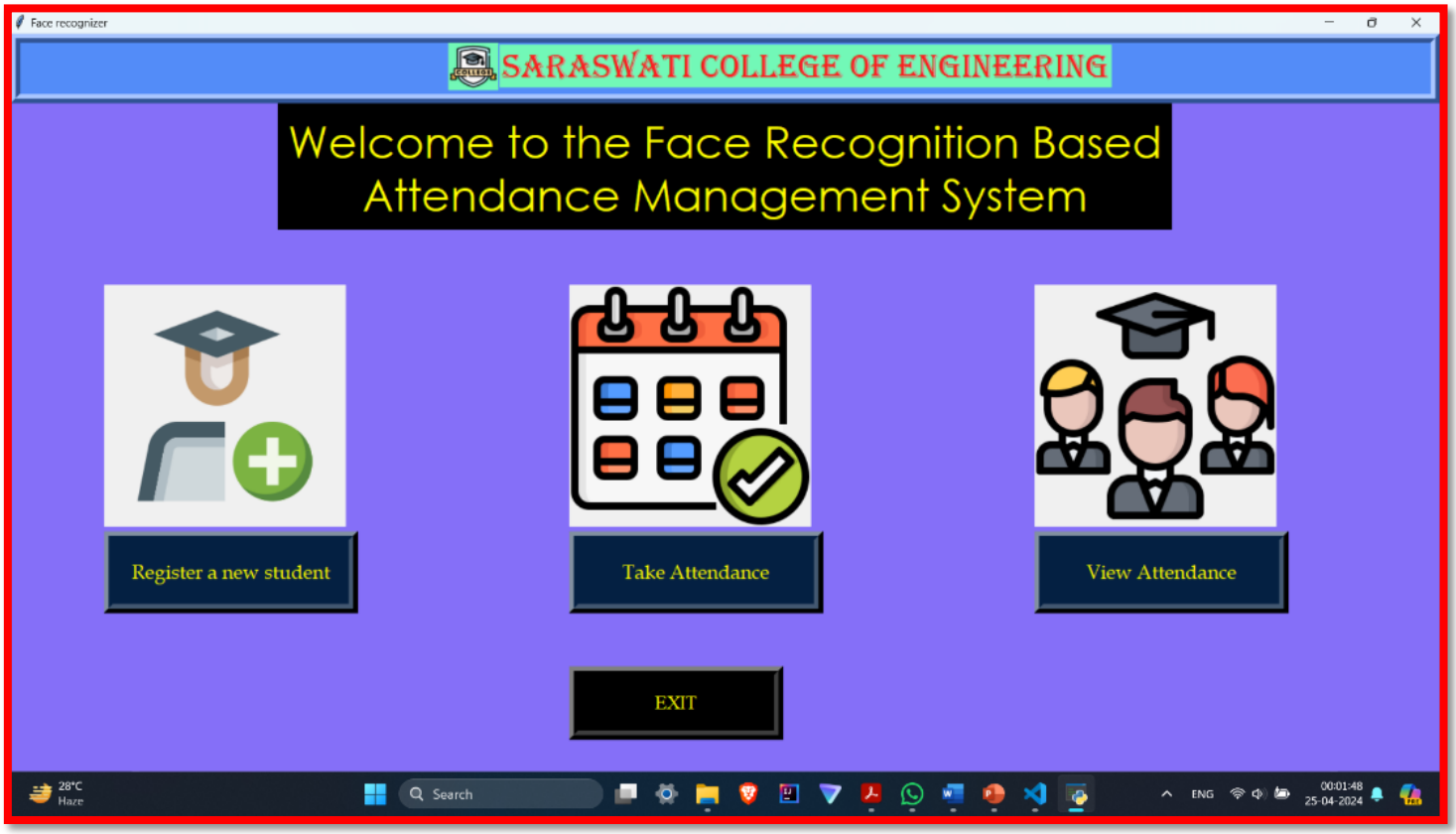


Fig 4: Login Page

Fig 5: Face Detection Home-Page



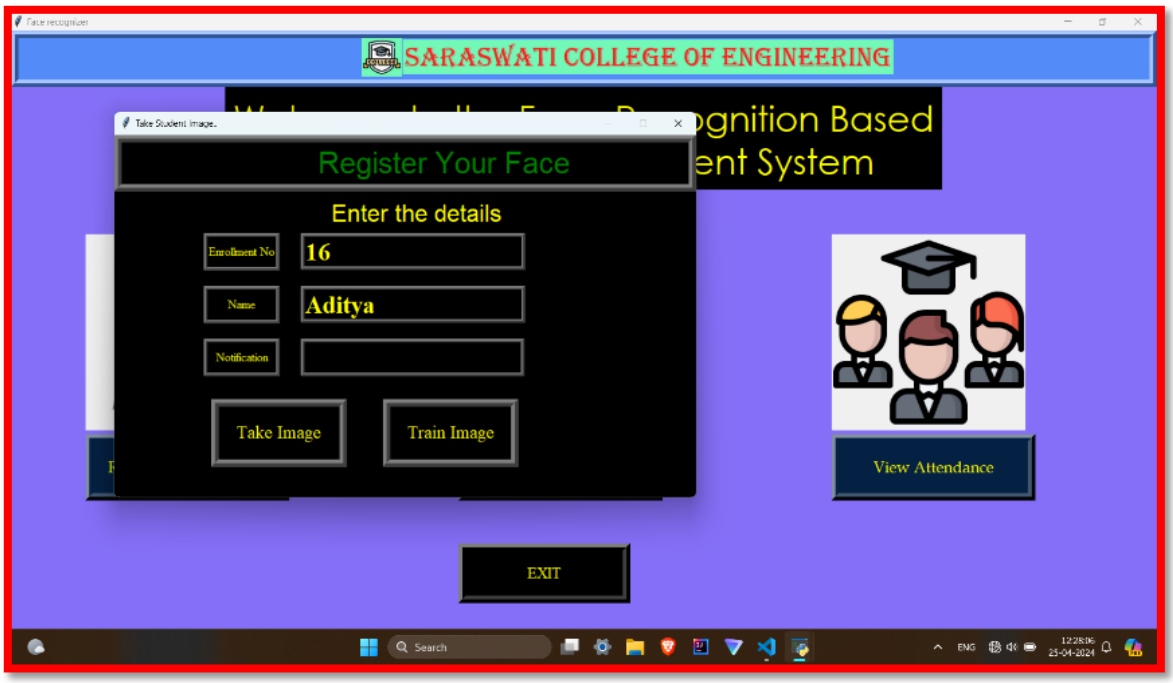
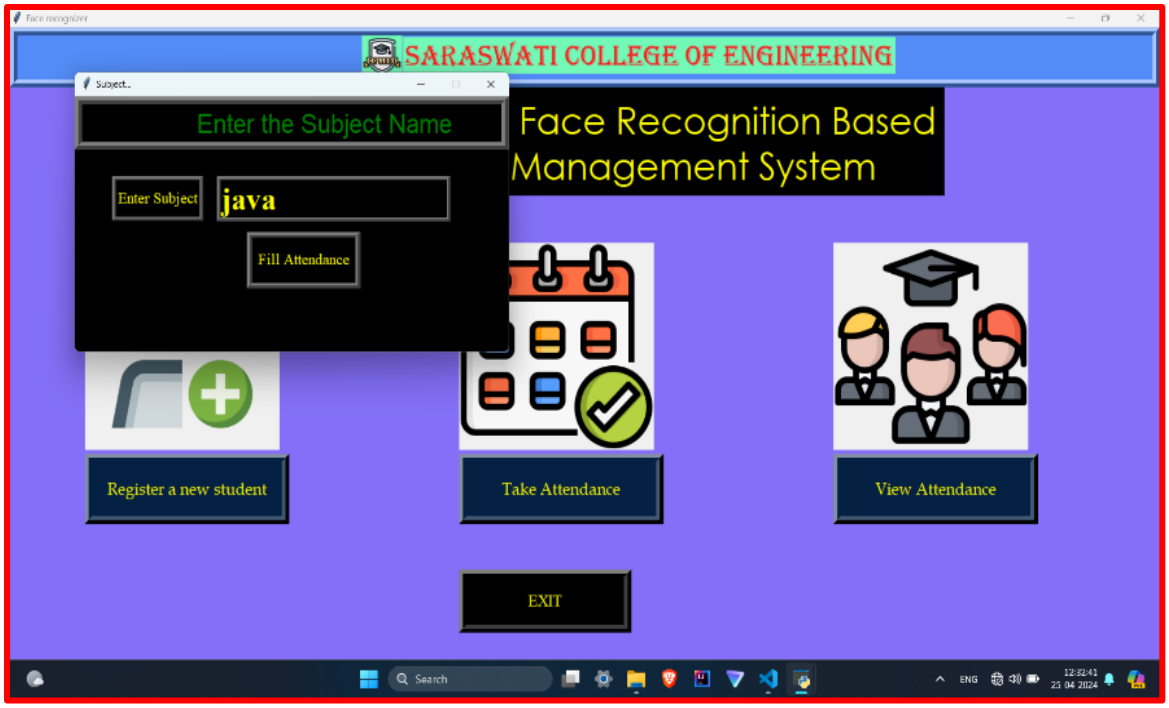


Fig 6: - Registering a New Student and Training Student Faces



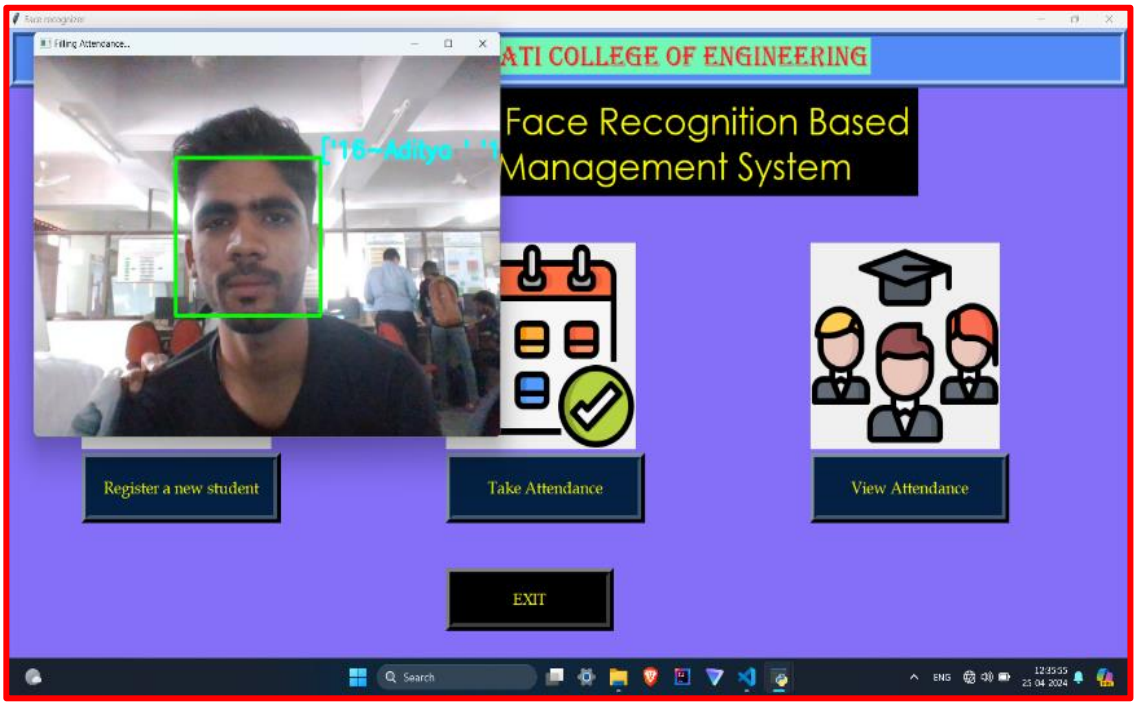
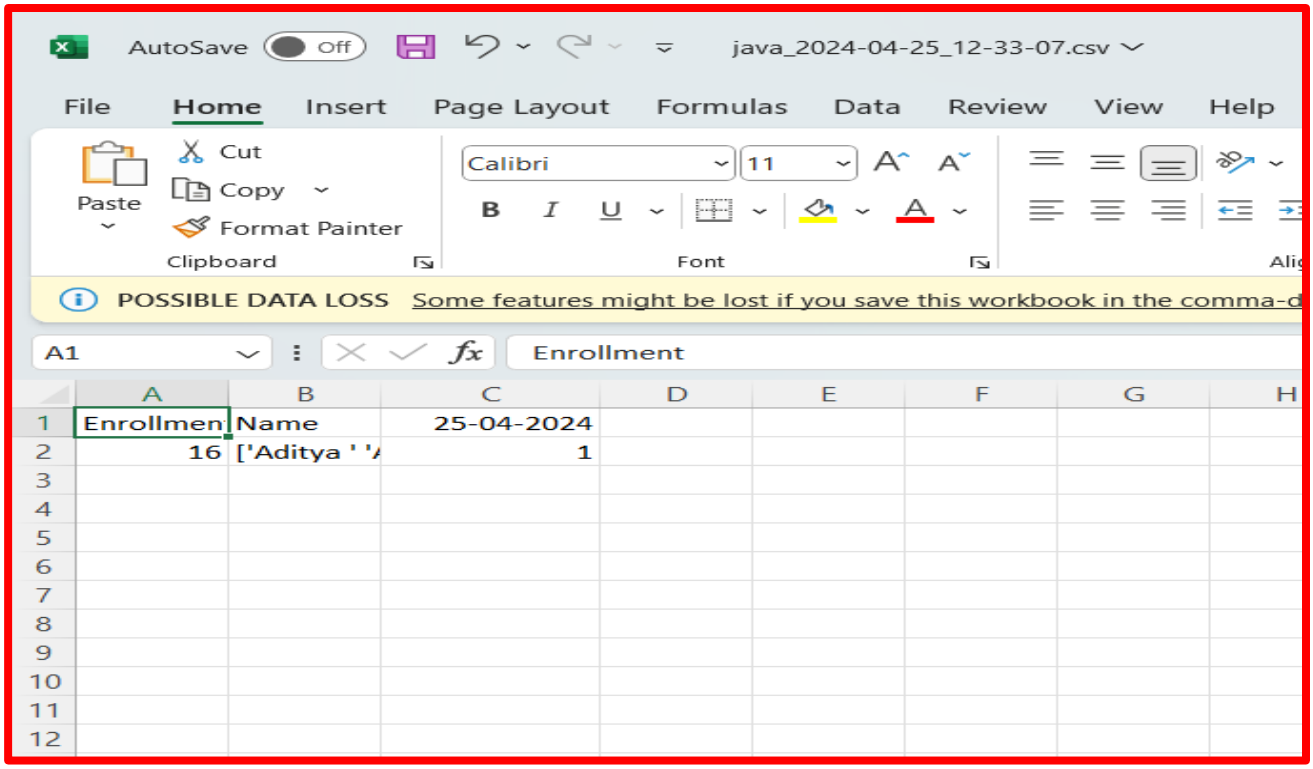


Fig 7: - Entering Subject and Recognizing Student Face through trained model of. yml



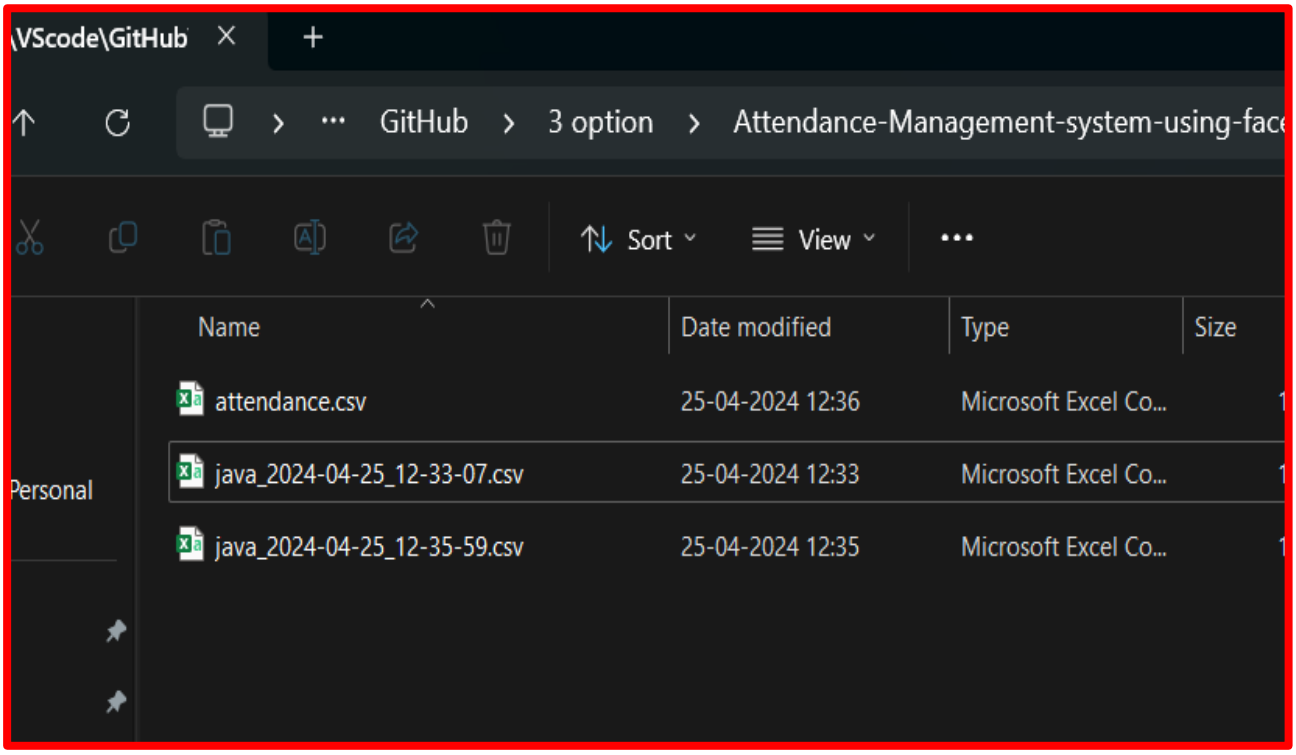


Fig 8: - Viewing Attendance and making Entry in the Excel Sheet.

CHAPTER 4

CONCLUSION AND FUTURE SCOPE

Conclusion: The Facial Detection Attendance System in Python has successfully demonstrated the potential of facial recognition technology in automating attendance processes. By utilizing advanced image processing and machine learning algorithms, the system has been able to accurately detect and identify individuals within a given environment. The implementation of this system has led to increased efficiency, reduced human error, and enhanced security in managing attendance records.

The project's success can be attributed to the following factors:

1. User-friendly interface: The system's easy-to-use interface allows users to interact with the software seamlessly, making it accessible to people with varying levels of technical expertise.
2. High accuracy: The facial recognition algorithms employed in the system have proven to be highly accurate in identifying individuals, ensuring that the attendance records are reliable.
3. Time-saving: The automated nature of the system has significantly reduced the time required for manual attendance recording, allowing for more efficient management of resources.
4. Security: By using facial recognition, the system helps prevent unauthorized access and maintains the confidentiality of attendance records.

Future Scope

Despite the success of the Facial Detection Attendance System in Python, there are several areas where improvements and expansions can be made, such as:

1. Integration with other systems: The system can be integrated with other software applications, such as payroll systems, to automate the process of calculating students working hours and generating salary slips.
2. Enhanced security features: Implementing additional security measures, such as multi-factor authentication or biometric verification, can further strengthen the system's security.
3. Improved accuracy: Continuous research and development can lead to the enhancement of facial recognition algorithms, resulting in even higher accuracy rates and better performance in diverse environments and lighting conditions.
4. Scalability: The system can be scaled to accommodate larger organizations or institutions, ensuring that it remains efficient and effective even as the number of users increases.
5. Multiple Face Detection: The future of multiple face detection involves optimizing algorithms for real-time processing, enabling applications to detect and track multiple faces instantaneously. This capability is crucial for security systems, surveillance, and interactive applications.
6. Artificial Intelligence (AI) integration: Incorporating AI technologies like deep learning and natural language processing can further improve the system's performance and introduce new functionalities, such as voice-based attendance or personalized recommendations for time management.

CHAPTER 5

REFERENCES:

OpenCV (Open-Source Computer Vision Library): This is a popular computer vision and machine learning software library. It provides various tools for face detection and recognition, which can be used for attendance systems. You can download it from their official website: https://opencv.org/

1. Tutorials and Examples:
    - OpenCV Python Tutorials: https://docs.opencv.org/master/d3/d52/tutorial_python_introduction.html
    - PythonCV Tutorials: https://pythoncv.org/tutorial/index.html
    - A step-by-step tutorial on implementing a face detection attendance system using PythonCV: https://www.codeproject.com/Tips/1225795/Face-Detection-Attendance-System-Using-PythonCV
    - Research Paper: https://ieeexplore.ieee.org/document/9725638
    - https://www.researchgate.net/publication/355886757_Face_Detection_and_Recognition_Using_OpenCV